LEZIONI 26-29
Adequacy of the Tests: Control Flow Coverage Criteria and frameworks

Ingegneria del Software 2 – Software Testing

Università degli Studi di Tor Vergata - Roma

Guglielmo De Angelis
guglielmo.deangelis@iasi.cnr.it

adeguatezza

- il nuovo problema da affrontare
  - decidere se/quando smettere di testare un SUT
- le nuove domande cui rispondere
  - i miei test sono sufficientemente buoni?
  - il mio SUT è stato testato meticolosamente?
  - i miei test sono adeguati per gli obiettivi prefissati?

adeguatezza – quali tecniche ?!?!

- le tecniche sono molteplici e si caratterizzano in base:
  - al tipo di processo
  - al tipo di artefatti che si usano/assumono
  - al tipo di strategie applicate
- in generale
  - non esiste una tecnica perfetta e generale
    - il testing è sempre una attività “best-effort”
  - non esiste una tecnica “migliore”
    - i risultati dipendono dai contesti
    - ci sono sempre aspetti positivi e negativi da valutare
    - è spesso necessario negoziare dei trade-off

functional VS structural





functional



- basato su modelli o specifiche del SUT non direttamente eseguite/eseguibili
- considerare i comportamenti osservabili
- non scopre errori dovuti a dettagli implementativi



functional – requirements

Apache BookKeeper™ - The BookKeeper protocol - Mozilla Firefox

Visualizza Cronologia Segnalibri Strumenti Aiuto

bookkeeper.apache.org/docs/4.10.0/development/protocol/

There are only E distinct write quorums in any ensemble. If $Q_w = Q_a$, then there is only one, as no striping occurs.

Ack quorums

The ack quorum for an entry is any subset of the write quorum of size Q_a. If Q_a bookies acknowledge an entry, it means it has been fully replicated.

Guarantees

The system can tolerate $Q_a - 1$ failures without data loss.

Bookkeeper guarantees that:

1. All updates to a ledger will be read in the same order as they were written.
2. All clients will read the same sequence of updates from the ledger.

Writing to ledgers

The BookKeeper protocol

- Ledgers
  - Ledger metadata
  - Ensembles
  - Write quorums
  - Ack quorums
  - Guarantees
- Writing to ledgers
  - Closing a ledger as a writer
  - Closing a ledger as a reader
- Ledgers to logs



functional – requirements

Apache BookKeeper™ - The BookKeeper protocol - Mozilla Firefox

Visualizza Cronologia Segnalibri Strumenti Aiuto

bookkeeper.apache.org/docs/4.10.0/development/protocol/

There are only **E** distinct write quorums in any ensemble. If $Q_w = Q_a$, then there is only one, as no striping occurs.

Ack quorums

The ack quorum for an entry is any subset of the write quorum of size Q_a. If Q_a bookies acknowledge an entry, it means it has been fully replicated.

Guarantees

The system can tolerate $Q_a - 1$ failures without data loss.

Bookkeeper guarantees that:

1. All updates to a ledger will be read in the same order as they were written.
2. All clients will read the same sequence of updates from the ledger.

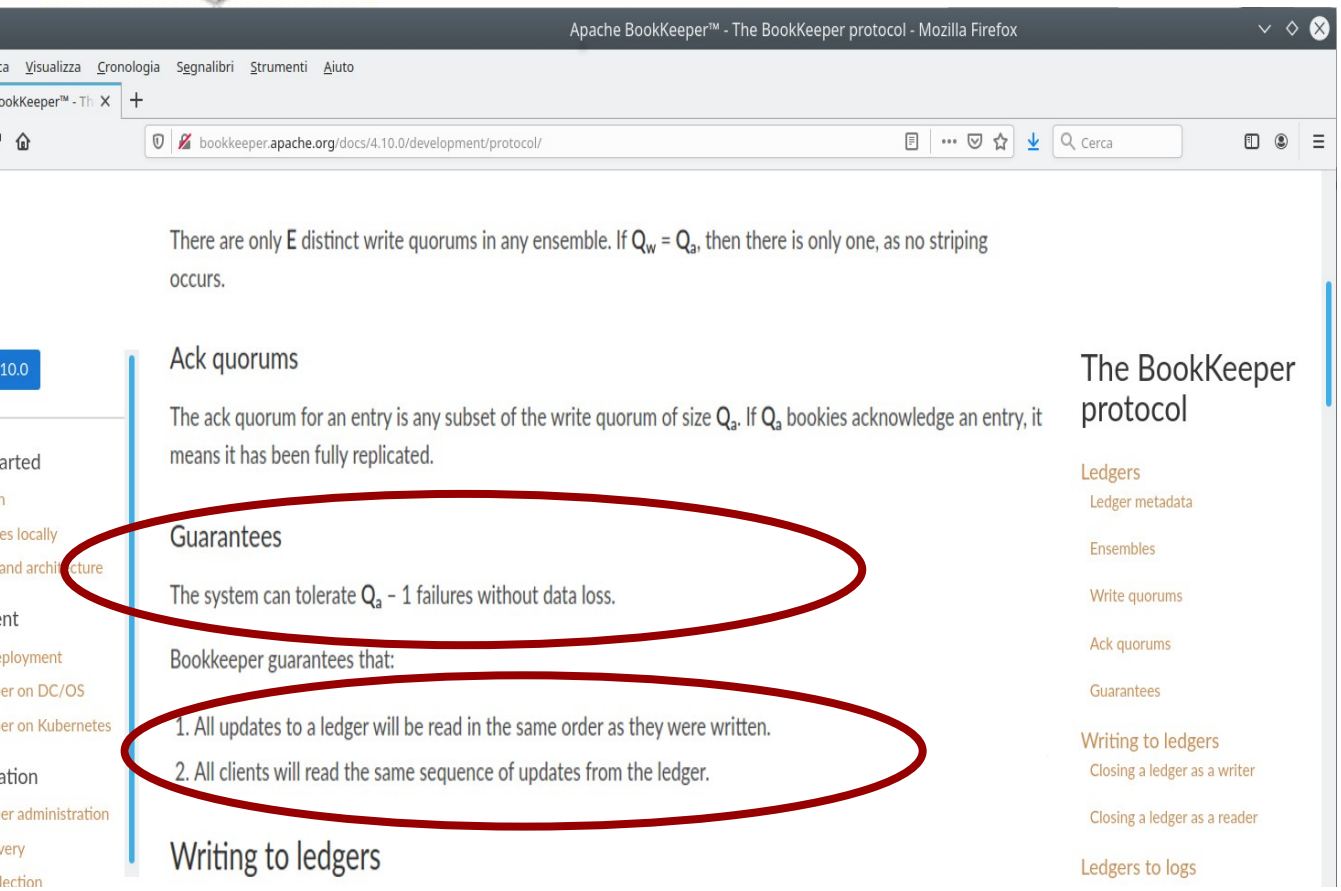
Writing to ledgers

The BookKeeper protocol

- Ledgers
  - Ledger metadata
  - Ensembles
  - Write quorums
  - Ack quorums
  - Guarantees
- Writing to ledgers
  - Closing a ledger as a writer
  - Closing a ledger as a reader
- Ledgers to logs



functional – requirements

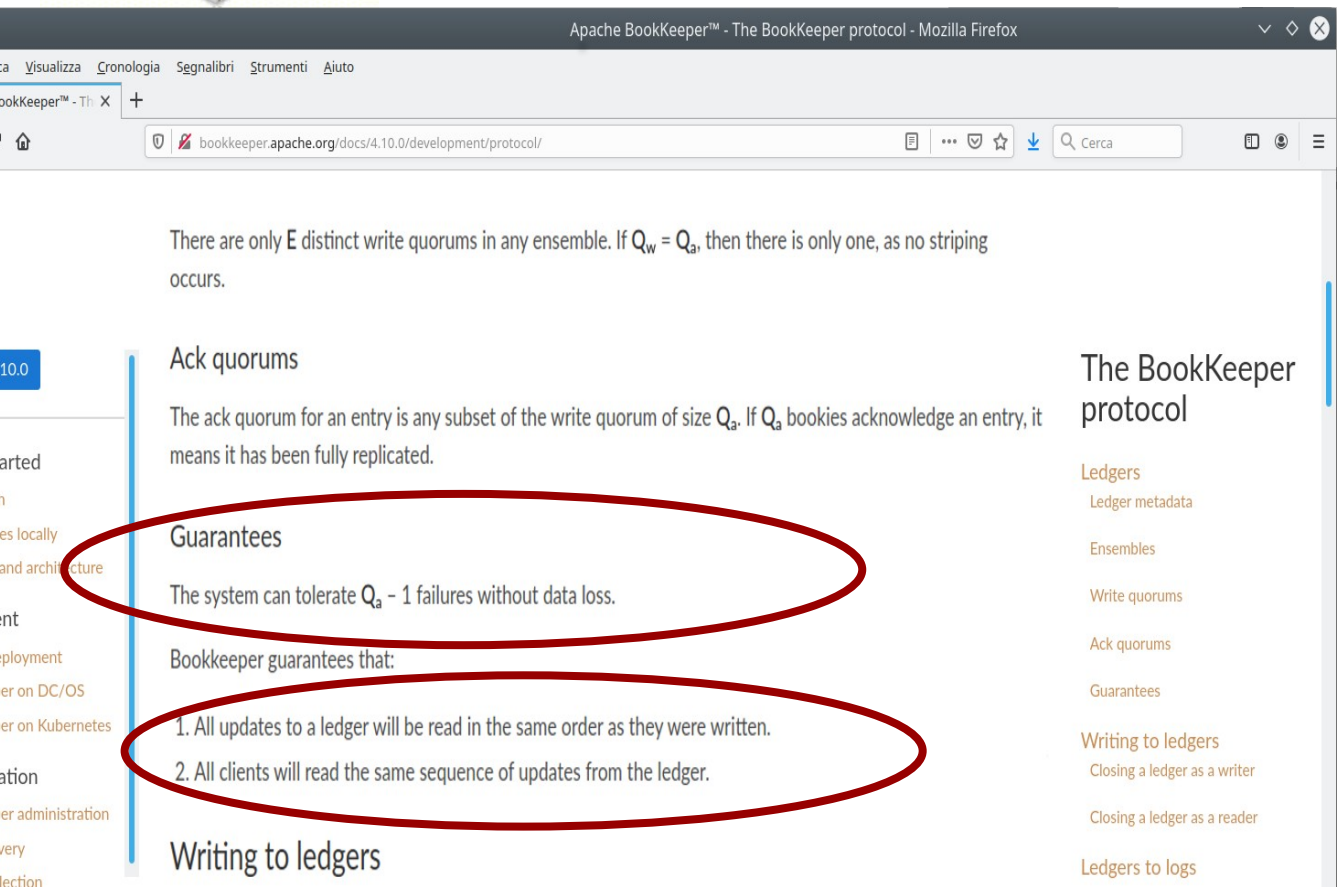


- CF1: almeno un test di scrittura-lettura su una entry dove nessun bookie fallisce
- CF2: almeno un test di scrittura-lettura su una entry che simuli il fallimento di Q_a-2 bookies
- CF3: almeno un test di scrittura-lettura su una entry che simuli il fallimento di Q_a-1 bookies
- CF4: almeno un test che scriva entries su un ledger e le recuperi controllando che siano nello stesso ordine
- CF5: almeno un test con un writer e più reader. Ad esempio il writer scrive entries in un dato ordine tutti i reader leggono. Ad ogni lettura i reader confrontano le entry recuperate

CFx: **C**riterio di Adeguatezza **F**unzionale **x**



functional – requirements

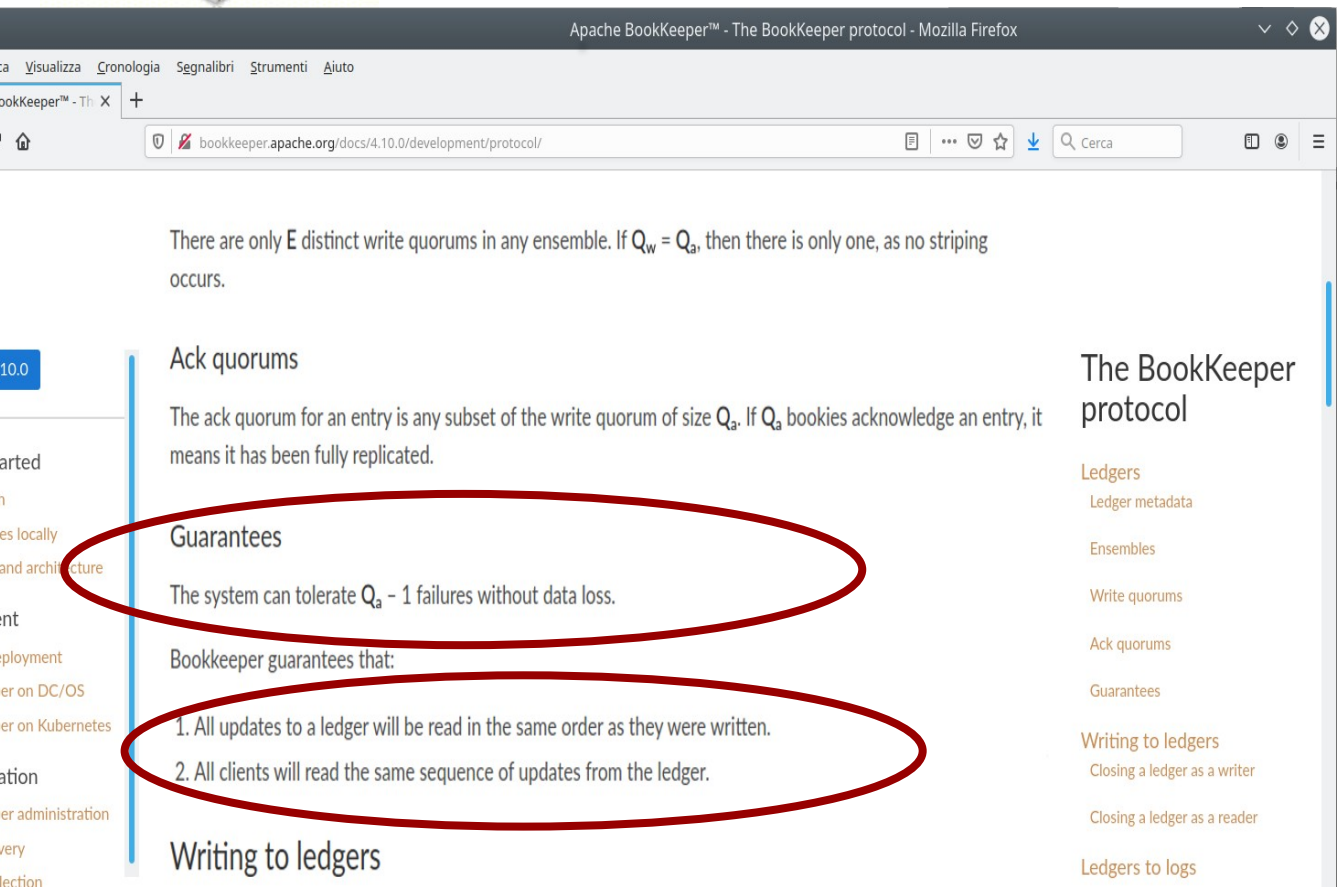


- CF1: almeno un test di scrittura-lettura su una entry dove nessun bookie fallisce
- CF2: almeno un test di scrittura-lettura su una entry che simuli il fallimento di Q_a-2 bookies
- CF3: almeno un test di scrittura-lettura su una entry che simuli il fallimento di Q_a-1 bookies
- CF4: almeno un test che scriva entries su un ledger e le recuperi controllando che siano nello stesso ordine
- CF5: almeno un test con un writer e più reader. Ad esempio il writer scrive entries in un dato ordine tutti i reader leggono. Ad ogni lettura i reader confrontano le entry recuperate

CFx: **C**riterio di Adeguatezza **F**unzionale **x**



functional – requirements

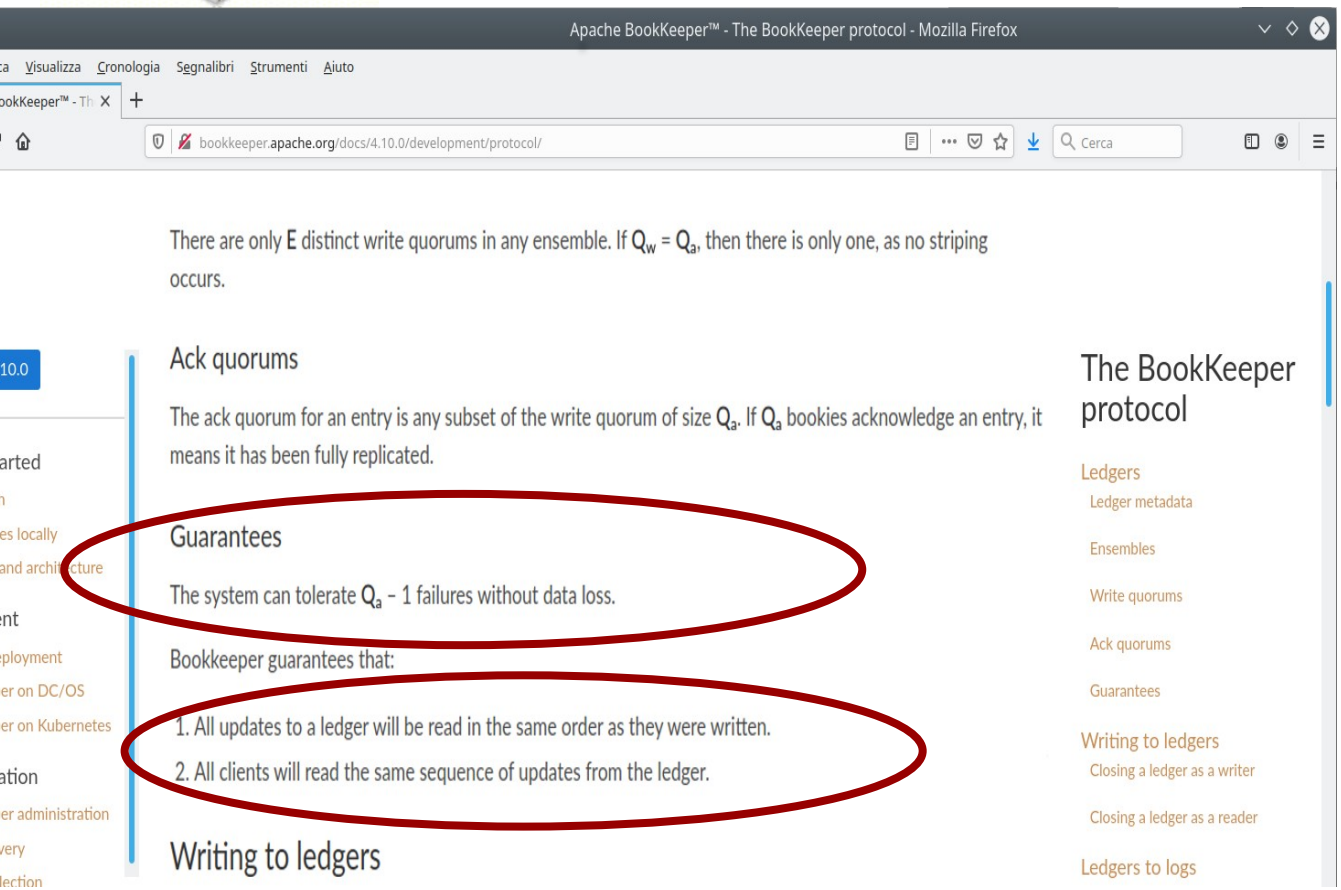


- CF1: almeno un test di scrittura-lettura su una entry dove nessun bookie fallisce
- CF2: almeno un test di scrittura-lettura su una entry che simuli il fallimento di Q_a-2 bookies
- CF3: almeno un test di scrittura-lettura su una entry che simuli il fallimento di Q_a-1 bookies
- CF4: almeno un test che scriva entries su un ledger e le recuperi controllando che siano nello stesso ordine
- CF5: almeno un test con un writer e più reader. Ad esempio il writer scrive entries in un dato ordine tutti i reader leggono. Ad ogni lettura i reader confrontano le entry recuperate

CFx: **C**riterio di Adeguatezza **F**unzionale **x**



functional – requirements

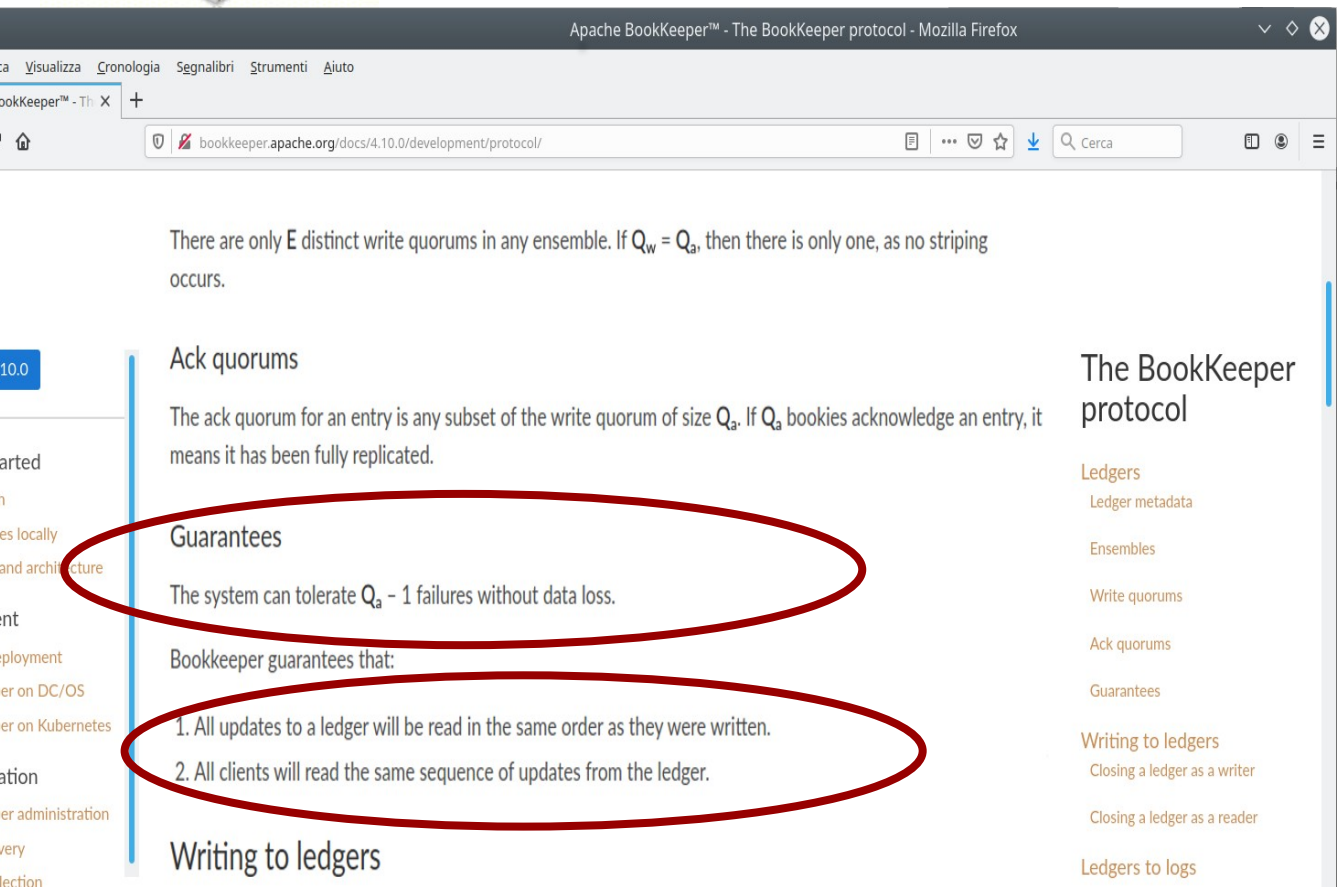


- CF1: almeno un test di scrittura-lettura su una entry dove nessun bookie fallisce
- CF2: almeno un test di scrittura-lettura su una entry che simuli il fallimento di Q_a-2 bookies
- CF3: almeno un test di scrittura-lettura su una entry che simuli il fallimento di Q_a-1 bookies
- CF4: almeno un test che scriva entries su un ledger e le recuperi controllando che siano nello stesso ordine
- CF5: almeno un test con un writer e più reader. Ad esempio il writer scrive entries in un dato ordine tutti i reader leggono. Ad ogni lettura i reader confrontano le entry recuperate

CFx: **C**riterio di Adeguatezza **F**unzionale **x**



functional – requirements

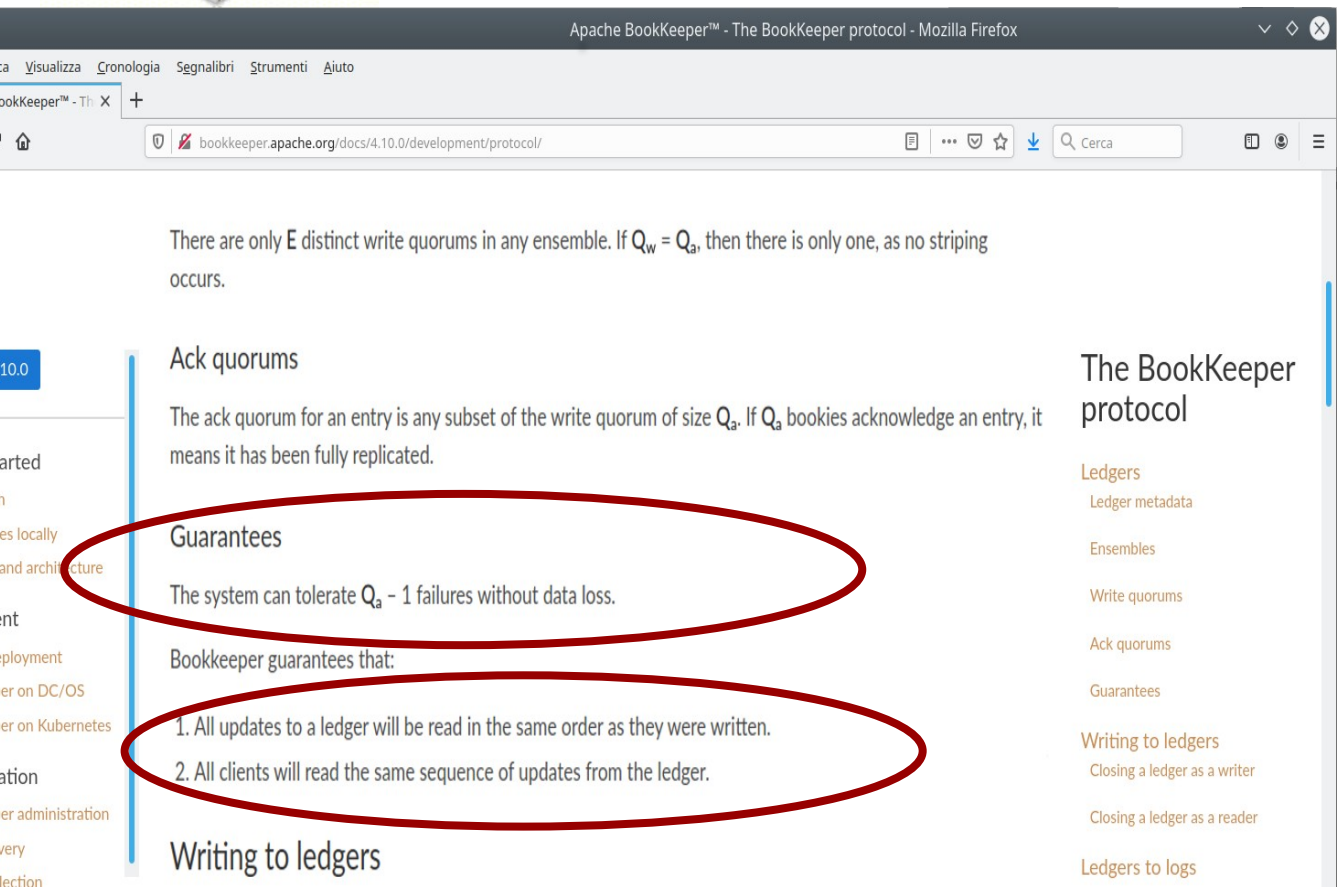


- CF1: almeno un test di scrittura-lettura su una entry dove nessun bookie fallisce
- CF2: almeno un test di scrittura-lettura su una entry che simuli il fallimento di Q_a-2 bookies
- CF3: almeno un test di scrittura-lettura su una entry che simuli il fallimento di Q_a-1 bookies
- CF4: almeno un test che scriva entries su un ledger e le recuperi controllando che siano nello stesso ordine
- CF5: almeno un test con un writer e più reader. Ad esempio il writer scrive entries in un dato ordine tutti i reader leggono. Ad ogni lettura i reader confrontano le entry recuperate

CFx: **C**riterio di Adeguatezza **F**unzionale **x**



functional – requirements



- CF1: almeno un test di scrittura-lettura su una entry dove nessun bookie fallisce
- CF2: almeno un test di scrittura-lettura su una entry che simuli il fallimento di Q_a-2 bookies
- CF3: almeno un test di scrittura-lettura su una entry che simuli il fallimento di Q_a-1 bookies
- CF4: almeno un test che scriva entries su un ledger e le recuperi controllando che siano nello stesso ordine
- CF5: almeno un test con un writer e più reader. Ad esempio il writer scrive entries in un dato ordine tutti i reader leggono. Ad ogni lettura i reader confrontano le entry recuperate

CFx: **C**riterio di Adeguatezza **F**unzionale **x**



functional – specs

Apache BookKeeper™ - The BookKeeper protocol - Mozilla Firefox

Segnalibri Strumenti Aiuto

bookkeeper.apache.org/docs/4.10.0/development/protocol/

Cerca

If a node fails to acknowledge a write, the writer will create a new ensemble by replacing the failed node in the current ensemble. It creates a new fragment with this ensemble, starting from the first message that has not been acknowledged to the client. Creating the new fragment involves making a CAS write to the metadata. If the CAS write fails, someone else has modified something in the ledger metadata. This concurrent modification could have been caused by recovery or rereplication. We reread the metadata. If the state of the ledger is no longer `OPEN`, we send an error to the client for any outstanding writes. Otherwise, we try to replace the failed node again.

Closing a ledger as a writer

Closing a ledger is straightforward for a writer. The writer makes a CAS write to the metadata, changing the state to `CLOSED` and setting the last entry of the ledger to the last entry which we have acknowledged to the client.

If the CAS write fails, it means someone else has modified the metadata. We reread the metadata, and retry closing as long as the state of the ledger is still `OPEN`. If the state is `IN_RECOVERY` we send an error to the client. If the state is `CLOSED` and the last entry is the same as the last entry we have acknowledged to the client, we complete the close operation successfully. If the last entry is different from what we have acknowledged to the client, we send an error to the client.

Closing a ledger as a reader

A reader can also force a ledger to close. Forcing the ledger to close will prevent any writer from adding new entries to the ledger. This is called fencing. This can occur when a writer has crashed or become unavailable, and a new writer wants to take over writing to the log. The new writer must ensure that it has seen all updates from the previous writer, and prevent the previous writer from making any new updates before making any updates of its own.

The BookKeeper protocol

Apache BookKeeper™ - The BookKeeper protocol - Mozilla Firefox

File Modifica Visualizza Cronologia Segnalibri Strumenti Aiuto

Apache BookKeeper™ - Th

bookkeeper.apache.org/docs/4.10.0/development/protocol/

Cerca

Version: 4.10.0

Getting started
- Installation
- Run bookies locally
- Concepts and architecture

Deployment
- Manual deployment
- BookKeeper on DC/OS
- BookKeeper on Kubernetes

Administration
- BookKeeper administration
- AutoRecovery
- Metric collection
- Upgrade
- BookKeeper Admin REST API
- Decommissioning Bookies

API
- Overview
- Ledger API

A ledger has a single writer and multiple readers (SWMR).

Ledger metadata

A ledger's metadata contains the following:

Parameter	Name	Meaning
Identifer		A 64-bit integer, unique within the system
Ensemble size	E	The number of nodes the ledger is stored on
Write quorum size	Q_w	The number of nodes each entry is written to. In effect, the max replication for the entry.
Ack quorum size	Q_a	The number of nodes an entry must be acknowledged on. In effect, the minimum replication for the entry.
Current state		The current status of the ledger. One of `OPEN`, `CLOSED`, or `IN_RECOVERY`.
Last entry		The last entry in the ledger or `NULL` is the current state is not `CLOSED`.

In addition, each ledger's metadata consists of one or more *fragments*. Each fragment is either

- the first entry of a fragment or
- a list of bookies for the fragment.

When creating a ledger, the following invariant must hold:

$E >= Q_w >= Qa$

Thus, the ensemble size (**E**) must be larger than the write quorum size (Q_w), which must in turn be larger than the ack quorum size (Q_a). If that condition does not hold, then the ledger creation operation will fail.

The BookKee protocol

Ledgers
- Ledger metadata
- Ensembles
- Write quorums
- Ack quorums
- Guarantees

Writing to ledgers
- Closing a ledger as a writ
- Closing a ledger as a read

Ledgers to logs
- Opening a log
- Rolling ledgers



functional – specs

Apache BookKeeper™ - The BookKeeper protocol - Mozilla Firefox

Segnalibri Strumenti Aiuto

bookkeeper.apache.org/docs/4.10.0/development/protocol/

If a node fails to acknowledge a write, the writer will create a new ensemble by replacing the failed node in the current ensemble. It creates a new fragment with this ensemble, starting from the first message that has not been acknowledged to the client. Creating the new fragment involves making a CAS write to the metadata. If the CAS write fails, someone else has modified something in the ledger metadata. This concurrent modification could have been caused by recovery or rereplication. We reread the metadata. If the state of the ledger is no longer `OPEN`, we send an error to the client for any outstanding writes. Otherwise, we try to replace the failed node again.

Closing a ledger as a writer

Closing a ledger is straightforward for a writer. The writer makes a CAS write to the metadata, changing the state to `CLOSED` and setting the last entry of the ledger to the last entry which we have acknowledged to the client.

If the CAS write fails, it means someone else has modified the metadata. We reread the metadata, and retry closing as long as the state of the ledger is still `OPEN`. If the state is `IN_RECOVERY` we send an error to the client. If the state is `CLOSED` and the last entry is the same as the last entry we have acknowledged to the client, we complete the close operation successfully. If the last entry is different from what we have acknowledged to the client, we send an error to the client.

Closing a ledger as a reader

A reader can also force a ledger to close. Forcing the ledger to close will prevent any writer from adding new entries to the ledger. This is called fencing. This can occur when a writer has crashed or become unavailable, and a new writer wants to take over writing to the log. The new writer must ensure that it has seen all updates from the previous writer, and prevent the previous writer from making any new updates before making any updates of its own.

The BookKeeper protocol

Apache BookKeeper™ - The BookKeeper protocol - Mozilla Firefox

File Modifica Visualizza Cronologia Segnalibri Strumenti Aiuto

bookkeeper.apache.org/docs/4.10.0/development/protocol/

Version: 4.10.0

Getting started
- Installation
- Run bookies locally
- Concepts and architecture

Deployment
- Manual deployment
- BookKeeper on DC/OS
- BookKeeper on Kubernetes

Administration
- BookKeeper administration
- AutoRecovery
- Metric collection
- Upgrade
- BookKeeper Admin REST API
- Decommissioning Bookies

API
- Overview
- Ledger API

A ledger has a single writer and multiple readers (SWMR).

Ledger metadata

A ledger's metadata contains the following:

Parameter	Name	Meaning
Identifer		A 64-bit integer, unique within the system
Ensemble size	E	The number of nodes the ledger is stored on
Write quorum size	Q_w	The number of nodes each entry is written to. In effect, the max replication for the entry.
Ack quorum size	Q_a	The number of nodes an entry must be acknowledged on. In effect, the minimum replication for the entry.
Current state		The current status of the ledger. One of `OPEN`, `CLOSED`, or `IN_RECOVERY`.
Last entry		The last entry in the ledger or `NULL` is the current state is not `CLOSED`.

In addition, each ledger's metadata consists of one or more *fragments*. Each fragment is either

- the first entry of a fragment or
- a list of bookies for the fragment.

When creating a ledger, the following invariant must hold:

$E >= Q_w >= Qa$

Thus, the ensemble size (**E**) must be larger than the write quorum size (Q_w), which must in turn be larger than the ack quorum size (Q_a). If that condition does not hold, then the ledger creation operation will fail.

The BookKee protocol

Ledgers
- Ledger metadata
- Ensembles
- Write quorums
- Ack quorums
- Guarantees

Writing to ledgers
- Closing a ledger as a writ
- Closing a ledger as a read

Ledgers to logs
- Opening a log
- Rolling ledgers



functional – specs

Apache BookKeeper™ - The BookKeeper protocol - Mozilla Firefox

Segnalibri Strumenti Aiuto

bookkeeper.apache.org/docs/4.10.0/development/protocol/

If a node fails to acknowledge a write, the writer will create a new ensemble by replacing the failed node in the current ensemble. It creates a new fragment with this ensemble, starting from the first message that has not been acknowledged to the client. Creating the new fragment involves making a CAS write to the metadata. If the CAS write fails, someone else has modified something in the ledger metadata. This concurrent modification could have been caused by recovery or rereplication. We reread the metadata. If the state of the ledger is no longer `OPEN`, we send an error to the client for any outstanding writes. Otherwise, we try to replace the failed node again.

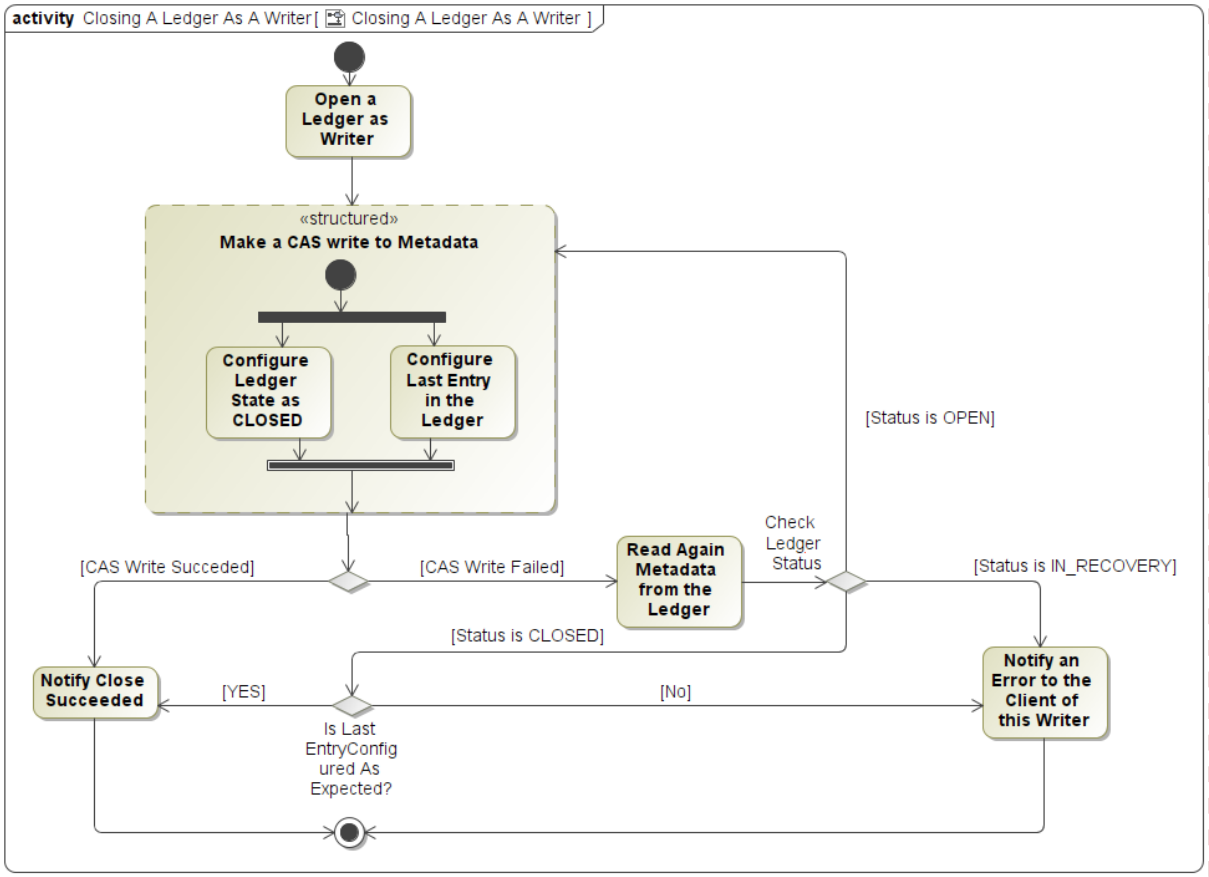
Closing a ledger as a writer

Closing a ledger is straightforward for a writer. The writer makes a CAS write to the metadata, changing the state to `CLOSED` and setting the last entry of the ledger to the last entry which we have acknowledged to the client.

If the CAS write fails, it means someone else has modified the metadata. We reread the metadata, and retry closing as long as the state of the ledger is still `OPEN`. If the state is `IN_RECOVERY` we send an error to the client. If the state is `CLOSED` and the last entry is the same as the last entry we have acknowledged to the client, we complete the close operation successfully. If the last entry is different from what we have acknowledged to the client, we send an error to the client.

Closing a ledger as a reader

A reader can also force a ledger to close. Forcing the ledger to close will prevent any writer from adding new entries to the ledger. This is called fencing. This can occur when a writer has crashed or become unavailable, and a new writer wants to take over writing to the log. The new writer must ensure that it has seen all updates from the previous writer, and prevent the previous writer from making any new updates before making any updates of its own.



Deployment
Manual deployment
BookKeeper on DC/OS
BookKeeper on Kubernetes
Administration
BookKeeper administration
AutoRecovery
Metric collection
Upgrade
BookKeeper Admin REST API
Decommissioning Bookies
API
Overview
Ledger API

Ack quorum size	Q_a	The number of nodes an entry must be acknowledged on. In effect, the minimum replication for this entry.
Current state	The current status of the ledger. One of `OPEN`, `CLOSED`, or `IN_RECOVERY`.
Last entry	The last entry in the ledger or `NULL` is the current state is not `CLOSED`.

In addition, each ledger's metadata consists of one or more *fragments*. Each fragment is either

- the first entry of a fragment or
- a list of bookies for the fragment.

When creating a ledger, the following invariant must hold:

$E >= Q_w >= Qa$

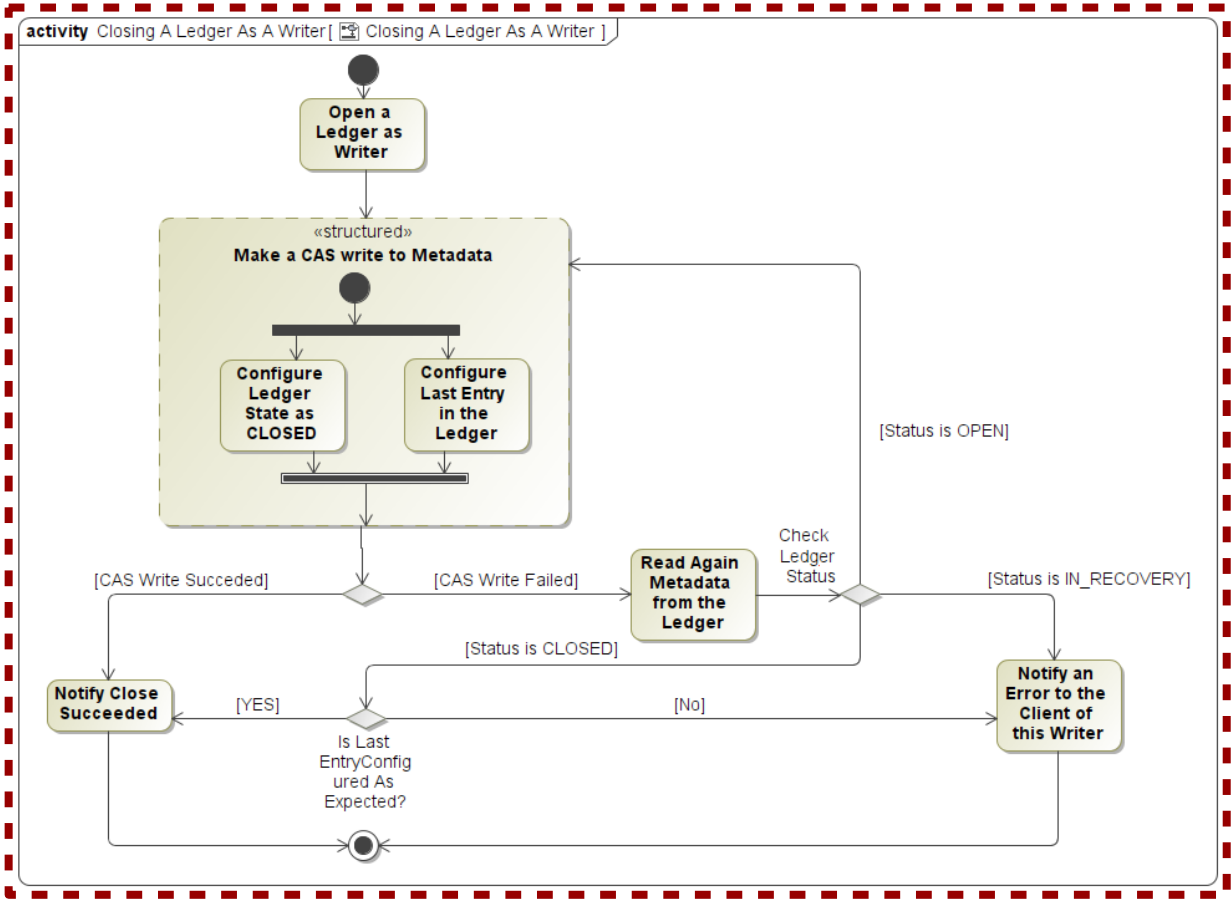
Thus, the ensemble size (**E**) must be larger than the write quorum size (Q_w), which must in turn be larger than the ack quorum size (Q_a). If that condition does not hold, then the ledger creation operation will fail.

Ack quorums
Guarantees
Writing to ledgers
Closing a ledger as a writ
Closing a ledger as a read
Ledgers to logs
Opening a log
Rolling ledgers



functional – specs

- altri esempi di criteri di adeguatezza:
  - CF6: almeno un test che si chiuda con successo
  - CF7: almeno un test che causi il fallimento del “CAS Write”
  - CF8: almeno un test che si chiuda con un errore



CFx: **C**riterio di Adeguatezza **F**unzionale **x**

functional - recap



- basato su modelli o specifiche del SUT non direttamente eseguite/eseguibili
- considerare i comportamenti osservabili
- non scopre errori dovuti a dettagli implementativi

structural



- basate sulla effettiva implementazione del SUT che verrà eseguita
  - su codice, o su altre astrazioni che forniscono una rappresentazione effettiva ed eseguibile del SUT
- considerare la maggior parte del codice possibile
- non può rivelare errori su funzionalità mancanti (e.g. missing paths)
- non c'è **nessuna garanzia** per cui alta coverage strutturale implichi l'esposizione di failures
  - nessun legame con le funzionalità o i modi d'uso
  - coverage strutturale impatta solo sulla nostra confidenza nell'esplorazione del SUT



structural

```
666 /**
667  * Read a sequence of entries asynchronously.
668  *
669  * @param firstEntry
670  *          id of first entry of sequence
671  * @param lastEntry
672  *          id of last entry of sequence
673  * @param cb
674  *          object implementing read callback interface
675  * @param ctx
676  *          control object
677  */
678 public void asyncReadEntries(long firstEntry, long lastEntry, ReadCallback cb, Object ctx) {
679     // Little sanity check
680     if (firstEntry < 0 || firstEntry > lastEntry) {
681         LOG.error("IncorrectParameterException on ledgerId:{} firstEntry:{} lastEntry:{}",
682                 ledgerId, firstEntry, lastEntry);
683         cb.readComplete(BKException.Code.IncorrectParameterException, this, null, ctx);
684         return;
685     }
686
687     if (lastEntry > lastAddConfirmed) {
688         LOG.error("ReadException on ledgerId:{} firstEntry:{} lastEntry:{}",
689                 ledgerId, firstEntry, lastEntry);
690         cb.readComplete(BKException.Code.ReadException, this, null, ctx);
691         return;
692     }
693
694     asyncReadEntriesInternal(firstEntry, lastEntry, cb, ctx, false);
695 }
```

esempi di criteri di adeguatezza

- CS1: ogni istruzione interessata da almeno un test
- CS2: almeno un test per ogni guardia booleana atomica a `true`
- CS3: almeno un test per ogni guardia booleana atomica a `false`

https://github.com/apache/bookkeeper/blob/e7430ce25e3609bf77b028e4f0475dad2147c22d/bookkeeper-server/src/main/java/org/apache/bookkeeper/client/LedgerHandle.java#L678

CSx: **C**riterio di Adeguatezza **S**trutturale **x**



structural

```
666 /**
667  * Read a sequence of entries asynchronously.
668  *
669  * @param firstEntry
670  *          id of first entry of sequence
671  * @param lastEntry
672  *          id of last entry of sequence
673  * @param cb
674  *          object implementing read callback interface
675  * @param ctx
676  *          control object
677  */
678 public void asyncReadEntries(long firstEntry, long lastEntry, ReadCallback cb, Object ctx) {
679     // Little sanity check
680     if (firstEntry < 0 || firstEntry > lastEntry) {
681         LOG.error("IncorrectParameterException on ledgerId:{} firstEntry:{} lastEntry:{}",
682                 ledgerId, firstEntry, lastEntry);
683         cb.readComplete(BKException.Code.IncorrectParameterException, this, null, ctx);
684         return;
685     }
686
687     if (lastEntry > lastAddConfirmed) {
688         LOG.error("ReadException on ledgerId:{} firstEntry:{} lastEntry:{}",
689                 ledgerId, firstEntry, lastEntry);
690         cb.readComplete(BKException.Code.ReadException, this, null, ctx);
691         return;
692     }
693
694     asyncReadEntriesInternal(firstEntry, lastEntry, cb, ctx, false);
695 }
```

esempi di criteri di adeguatezza

- CS1: ogni istruzione interessata da almeno un test
- CS2: almeno un test per ogni guardia booleana atomica a `true`
- CS3: almeno un test per ogni guardia booleana atomica a `false`

https://github.com/apache/bookkeeper/blob/e7430ce25e3609bf77b028e4f0475dad2147c22d/bookkeeper-server/src/main/java/org/apache/bookkeeper/client/LedgerHandle.java#L678

CSx: **C**riterio di Adeguatezza **S**trutturale **x**



structural

NOTA: in base alla struttura del codice,
alcuni criteri potrebbero non essere sempre soddisfacibili

vedere materiale allegato alla lezione:
`it.uniroma2.dicii.is2.trivialExamples.UnreachableCodeSimpleExample`

```
681         LOG.error("IncorrectParameterException on ledgerId:{} firstEntry:{} lastEntry:{}",
682                 ledgerId, firstEntry, lastEntry);
683         cb.readComplete(BKException.Code.IncorrectParameterException, this, null, ctx);
684         return;
685     }
686
687     if (lastEntry > lastAddConfirmed) {
688         LOG.error("ReadException on ledgerId:{} firstEntry:{} lastEntry:{}",
689                 ledgerId, firstEntry, lastEntry);
690         cb.readComplete(BKException.Code.ReadException, this, null, ctx);
691         return;
692     }
693
694     asyncReadEntriesInternal(firstEntry, lastEntry, cb, ctx, false);
695 }
```

eri di

ruzione
a almeno

un test per ogni guardia booleana atomica a `true`

- CS3: almeno un test per ogni guardia booleana atomica a `false`

https://github.com/apache/bookkeeper/blob/e7430ce25e3609bf77b028e4f0475dad2147c22d/bookkeeper-server/src/main/java/org/apache/bookkeeper/client/LedgerHandle.java#L678

CSx: **C**riterio di Adeguatezza **S**trutturale **x**

criteri di adeguatezza ...

- analizziamo nel dettaglio un insieme di criteri noti come *criteri di copertura*
  - criteri di copertura *black-box*: se dipendono esclusivamente dai requisiti o da una qualche specifica del SUT (e.g. un modello comportamentale del SUT)
  - criteri di copertura *white-box*: se dipendono esclusivamente dall'implementazione effettiva del SUT

criteri di adeguatezza ...

- analizziamo nel dettaglio un insieme di criteri noti come *criteri di copertura*
  - criteri di copertura *black-box*: se dipendono esclusivamente dai requisiti o da una qualche specifica del SUT (e.g. un modello comportamentale del SUT)
  - criteri di copertura *white-box*: se dipendono esclusivamente dall'implementazione effettiva del SUT

per esigenze di presentazione, gli esempi trattatti nel seguito faranno riferimento a criteri white-box. In base al modello funzionale adottato, è possibile definire analoghi criteri black-box (che non tratteremo esplicitamente)

analisi statica ed intuizione di control-flow graph

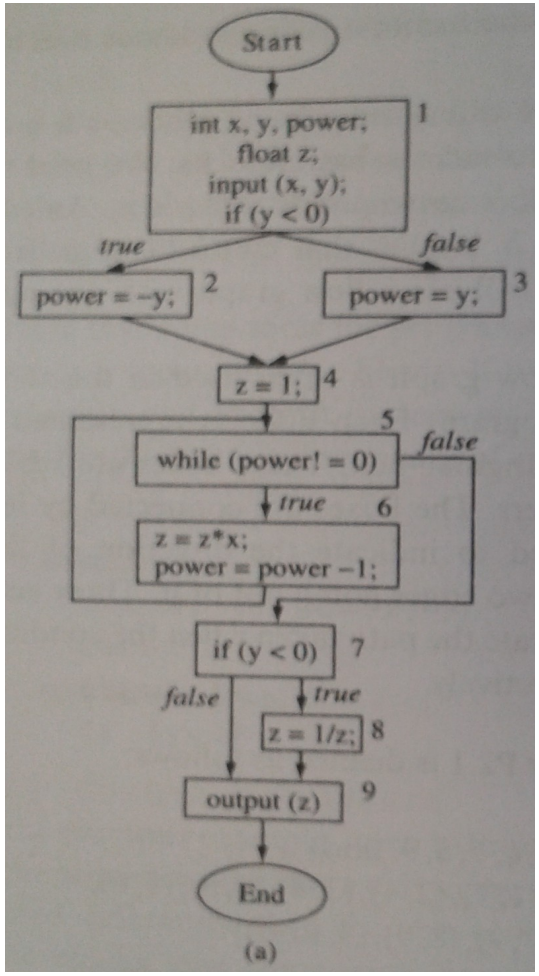
analisi statica ed intuizione di control-flow graph

```
1  begin
2    int x, y, power;
3    float z;
4    input (x, y);
5    if (y<0)
6     power = -y;
7    else
8     power = y;
9    z = 1;
10   while (power != 0){
11    z = z*x;
12    power = power-1;
13   }
14   if (y<0)
15    z = 1/z;
16   output(z);
17 end
```

analisi statica ed intuizione di control-flow graph

```
1   begin
2     int x, y, power;
3     float z;
4     input (x, y);
5     if (y<0)
6      power=-y;
7     else
8      power=y;
9     z=1;
10    while (power!=0){
11     z=z*x;
12     power=power-1;
13    }
14    if (y<0)
15     z=1/z;
16    output(z);
17  end
```

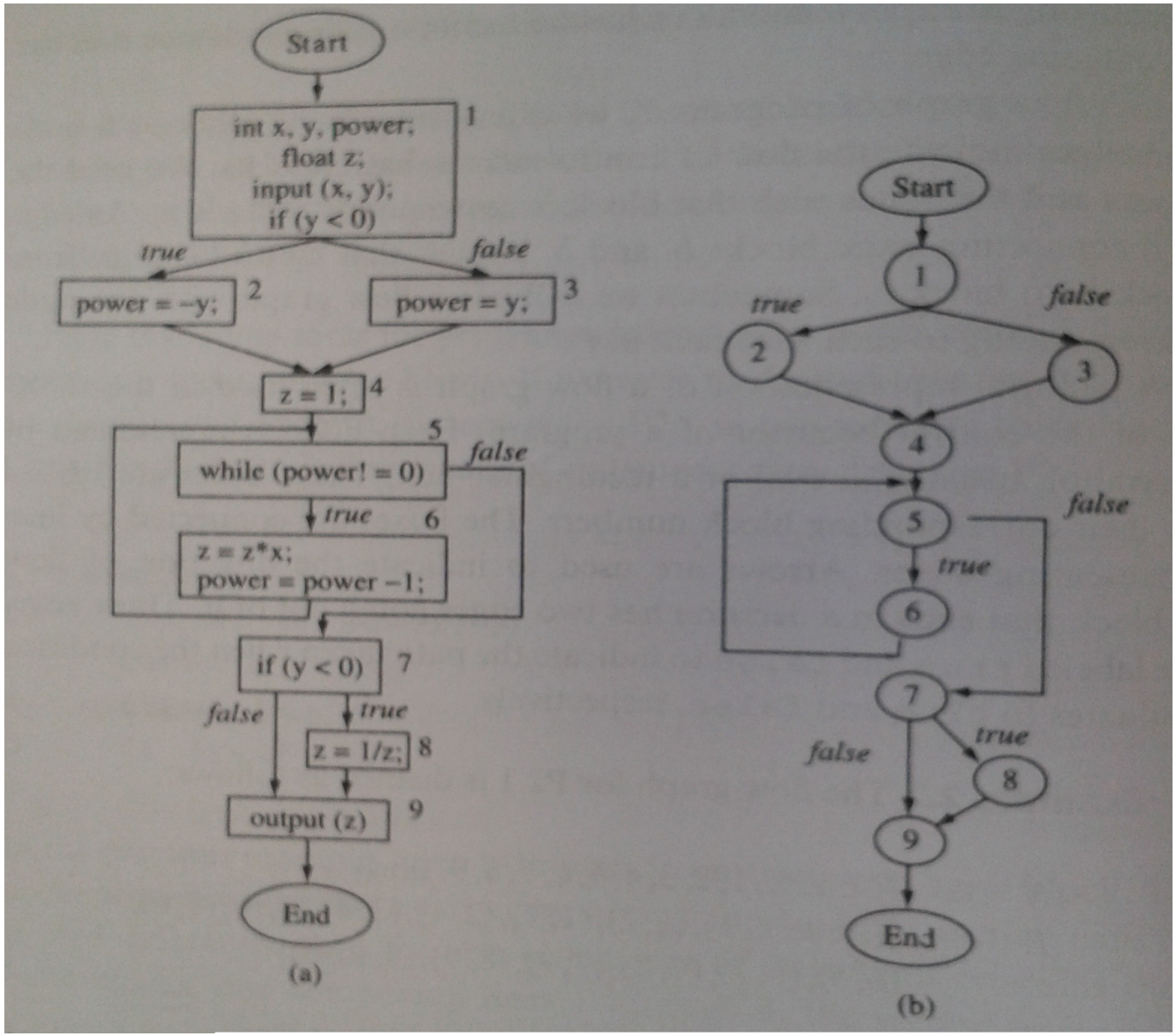
SEQUENZE DI ISTRUZIONI



NODI ED ARCHI

$$N = \{Start, 1, 2, 3, 4, 5, 6, 7, 8, 9, End\}$$
$$E = \{(Start, 1), (1, 2), (1, 3), (2, 4), (3, 4), (4, 5), (5, 6), (6, 5), (5, 7), (7, 8), (7, 9), (8, 9), (9, End)\}$$

analisi statica ed intuizione di control-flow graph



criteri basati su control-flow

- statement coverage: copertura dell'insieme di test T rispetto alle istruzioni presenti nel SUT
  - misura:

$$\frac{|S_c|}{(|S_e| - |S_i|)}$$

    - S_c: insieme di istruzioni coinvolte in almeno un test
    - S_e: insieme di istruzioni nel SUT
    - S_i: insieme di istruzioni (note come) non raggiungibili nel SUT
  - criteri:
    - T è adeguato per SUT se la sua statement coverage è 1
      - necessaria per implicazioni formali tra criteri di adeguatezza
    - T è accettabile per SUT se la sua statement coverage è almeno X%